Практична робота №1: HTTP URL Health Checker

Тема: Базові конструкції, структури даних та конкурентність у Go

---

📌 Опис проєкту

Консольний сервіс для перевірки доступності HTTP-ресурсів (Health Checker), написаний мовою Go. Проєкт демонструє поетапну еволюцію архітектури програми:

1. **Послідовне опитування** з використанням власних структур, інтерфейсів та HTTP-клієнта.
2. **Конкурентне опитування** за допомогою легковажних потоків (`goroutines`), примітивів синхронізації `sync.WaitGroup` та усунення стану гонитви через `sync.Mutex`.
3. **Асинхронний збір результатів через канали** (`channels`), безпечне керування життєвим циклом каналу горутиною-координатором та порівняльний аналіз швидкодії.

🎯 Мета роботи

• Набути практичних навичок роботи з системою типів мови Go: структурами (`struct`), методами з одержувачами (`receiver`), явним опрацюванням помилок (`error`).
• Опанувати принципи проєктування інтерфейсів (`interface`) та неявної імплементації (качиної типізації).
• Засвоїти фундаментальну парадигму конкурентності CSP: запуск легковажних потоків (`goroutine`), синхронізацію їхньої роботи через лічильники `sync.WaitGroup` та взаємне виключення `sync.Mutex`.
• Опанувати потокобезпечний обмін даними між горутинами за допомогою каналів (`chan`) та їхнє безпечне закриття.

🛠️ Технічне забезпечення

• **Операційна система:** Linux
• **Середовище розробки:** Neovim (gopls LSP)
• **Компілятор:** Go SDK 1.22+
• **Термінал / CLI:** bash

• **Система контролю версій:** Git
• **Утиліти діагностики:** вбудований детектор стану гонитви пам’яті (`go run -race`)

🚀 Інструкція із запуску

1. Перехід у робочу директорію

```
cd ~/Work/Health-service
```

2. Звичайний запуск програми

```
go run .
```

3. Запуск із вбудованим детектором стану гонитви (Race Detector)

Обов'язкова перевірка на відсутність race condition:

```
go run -race .
```

4. Збірка виконуваного бінарного файлу

```
go build -o health-checker .
./health-checker
```

🏗️ Етапи реалізації та архітектура

Рівень 1. Базовий: структури, методи та послідовний моніторинг

1. **Ініціалізація проєкту:**
   Створено окрему робочу директорію та ініціалізовано модуль:
   ```
   go mod init health/service
   ```
2. **Моделювання результатів:**
   Описано структуру `CheckResult` для збереження стану запиту:

```
type CheckResult struct {
    URL        string
    StatusCode int
    Duration   time.Duration
    Error      error
}
```

3. **Проєктування інтерфейсу та обробника:**
   Інтерфейс `Health` абстрагує операцію опитування, а `HTTPChecker` реалізує його через стандартний `net/http`:

```
type Health interface {
    CheckHealth(url string) CheckResult
}

type HTTPChecker struct {
    client *http.Client
}

func NewHTTPChecker(timeout time.Duration) *HTTPChecker {
    return &HTTPChecker{
        client: &http.Client{Timeout: timeout},
    }
}

func (c *HTTPChecker) CheckHealth(url string) CheckResult {
    start := time.Now()
    resp, err := c.client.Get(url)
    duration := time.Since(start)

    if err != nil {
        return CheckResult{
            URL:      url,
            Duration: duration,
            Error:    err,
        }
    }
    defer resp.Body.Close()

    return CheckResult{
        URL:        url,
        StatusCode: resp.StatusCode,
        Duration:   duration,
    }
}
```

4. **Послідовний прогін:**
   Опитування зрізу адрес у синхронному циклі. Запити блокують один одного, сумарний час роботи дорівнює сумі тривалості всіх запитів.

Рівень 2. Конкурентний: горутини та захист спільного стану через Mutex

1. **Розпаралелювання запитів:**
   Кожен URL перевіряється в окремій горутині через `go func(url string)`.
   Передача адреси параметром функції запобігає замиканню змінної циклу.
2. **Координація завершення:**
   Головний потік очікує виконання за допомогою `sync.WaitGroup` (`Add(1)` перед запуском, `defer wg.Done()` всередині, `wg.Wait()` перед виходом).
3. **Захист критичної секції:**
   Використання `sync.Mutex` (`Lock()` / `Unlock()`) для запобігання пошкодженню пам'яті під час виведення в консоль або модифікації спільних змінних.

Рівень 3. Просунутий: асинхронний збір результатів через канали

1. **Передача результатів через канал:**
   Замість блокування спільної пам'яті м'ютексом створено буферизований канал `chan CheckResult`. Горутини відправляють результати безпосередньо у канал:

```
results := make(chan CheckResult, len(urls))
for _, u := range urls {
    wg.Add(1)
    go func(url string) {
        defer wg.Done()
        results <- checker.CheckHealth(url)
    }(u)
}
```

2. **Горутина-координатор:**
   Окрема фонова горутина очікує завершення всіх обчислень і безпечно закриває канал:

```
go func() {
    wg.Wait()
    close(results)
}()
```

3. **Читання в головному потоці:**
   Головна функція вичитує всі результати з каналу через `for res := range results`, підраховує кількість успішних/неуспішних з'єднань і формує підсумковий звіт.

📊 Результати роботи та скриншоти

1. Перевірка на відсутність стану гонитви (Race Detector)

Запуск із прапорцем `-race` гарантує відсутність взаємних блокувань і несинхронізованого доступу до спільних ресурсів:

```
go run -race .
```



Лог виконання:

```
…/Work/Health-service ❱ go run -race .
=== HTTP URL Health Checker ===
[Конкурентна перевірка адрес...]
-> https://invalid-fake-host-test.org: Помилка (Get "https://invalid-fake-host-test.org": dial tcp: lookup invalid-fake-host-test.org: no such host)
-> https://github.com: Status 200 OK (час: 22ms)
-> https://httpbin.org/status/404: Status 404 Not Found (час: 110ms)
-> https://go.dev: Status 200 OK (час: 204ms)

==========
ПІДСУМКОВИЙ ЗВІТ МОНІТОРИНГУ:
- Усього перевірено адрес: 4
- Успішних з'єднань: 3
- Мережевих помилок: 1
- Загальний час роботи: 205ms (проти 1.015s у послідовному режимі)
==========
```

> **Висновок діагностики:** жодних попереджень `WARNING: DATA RACE` не зафіксовано. Робота з пам'яттю повністю безпечна.

---

2. Порівняльний аналіз швидкодії

Режим роботи	Механізм	Загальний час	Прискорення
Послідовний (Рівень 1)	Синхронний цикл `for`	~1015 ms	1.0x (базовий)
Паралельний (Рівень 3)	Горутини + канали	~205 ms	**~4.8x**

Паралельний підхід скорочує час виконання до тривалості найповільнішого окремого запиту замість суми всіх запитів.

Лістинг

```
package main

import (
    "fmt"
    "net/http"
    "sync"
    "time"
)

type CheckResult struct {
    URL        string
    StatusCode int
    Duration   time.Duration
    Error      error
}

type Health interface {
    CheckHealth(url string) CheckResult
}

type HTTPChecker struct {
    client *http.Client
}

func NewHTTPChecker(timeout time.Duration) *HTTPChecker {
    return &HTTPChecker{
        client: &http.Client{Timeout: timeout},
    }
}

func (c *HTTPChecker) CheckHealth(url string) CheckResult {
    start := time.Now()
    resp, err := c.client.Get(url)
    duration := time.Since(start)
```

```
    if err != nil {
        return CheckResult{
            URL:      url,
            Duration: duration,
            Error:    err,
        }
    }
    defer resp.Body.Close()

    return CheckResult{
        URL:        url,
        StatusCode: resp.StatusCode,
        Duration:   duration,
    }
}

func main() {
    urls := []string{
        "https://go.dev",
        "https://github.com",
        "https://httpbin.org/status/404",
        "https://invalid-fake-host-test.org",
    }

    checker := NewHTTPChecker(5 * time.Second)

    seqStart := time.Now()
    for _, u := range urls {
        checker.CheckHealth(u)
    }
    seqDuration := time.Since(seqStart)

    fmt.Println("=== HTTP URL Health Checker ===")
    fmt.Println("[Конкурентна перевірка адрес...]")

    parStart := time.Now()
    results := make(chan CheckResult, len(urls))
    var wg sync.WaitGroup

    for _, u := range urls {
        wg.Add(1)
        go func(url string) {
            defer wg.Done()
            results <- checker.CheckHealth(url)
        }(u)
    }

    go func() {
        wg.Wait()
        close(results)
```

```
    }()

    var success, fail int
    for res := range results {
        if res.Error != nil {
            fail++
            fmt.Printf("-> %s: Помилка (%v)\n", res.URL, res.Error)
        } else {
            success++
            fmt.Printf("-> %s: Status %d %s (час: %v)\n",
                res.URL,
                res.StatusCode,
                http.StatusText(res.StatusCode),
                res.Duration.Round(time.Millisecond),
            )
        }
    }
    parDuration := time.Since(parStart)

    fmt.Println()
    fmt.Println("==========================================")
    fmt.Println("ПІДСУМКОВИЙ ЗВІТ МОНІТОРИНГУ:")
    fmt.Printf("- Усього перевірено адрес: %d\n", len(urls))
    fmt.Printf("- Успішних з'єднань: %d\n", success)
    fmt.Printf("- Мережевих помилок: %d\n", fail)
    fmt.Printf("- Загальний час роботи: %v (проти %v у послідовному
режимі)\n",
        parDuration.Round(time.Millisecond),
        seqDuration.Round(time.Millisecond),
    )
    fmt.Println("==========================================")
}
```